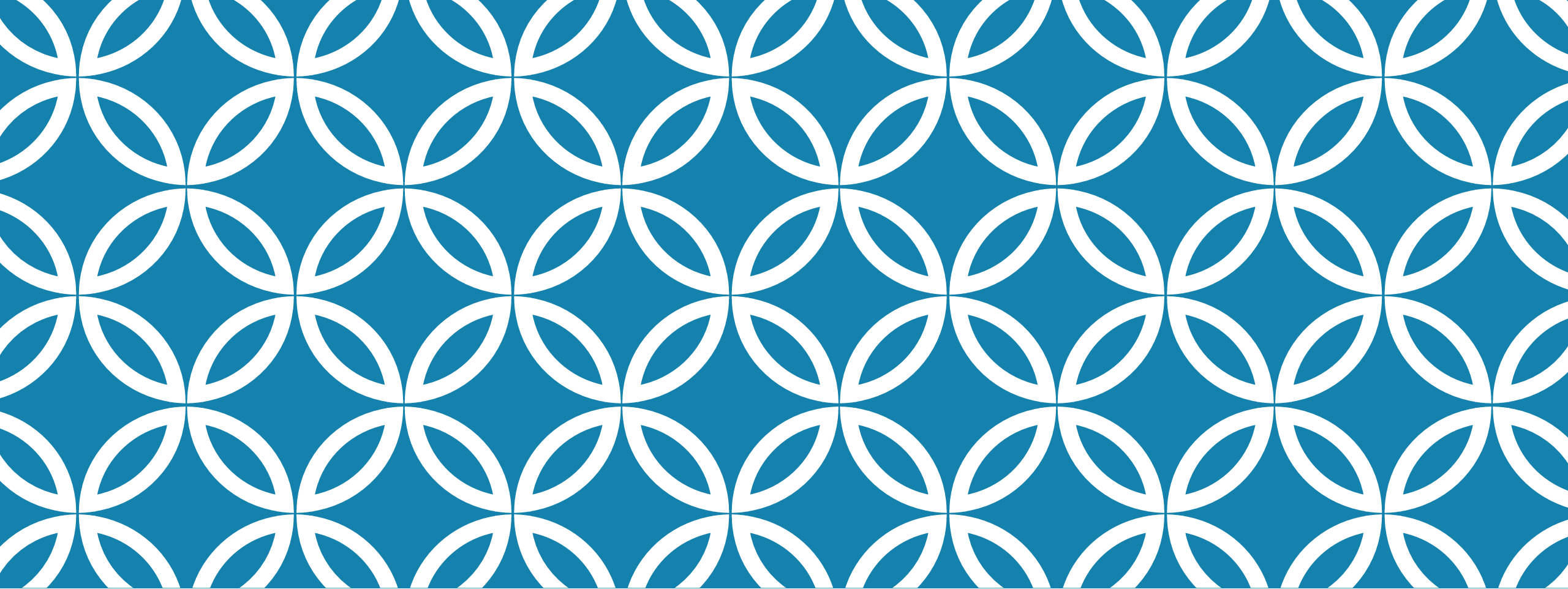




STATIC PROGRAM ANALYSIS

Lecture 08 – Formally defined
data-flow analyses (4 classical ones)

OPTIMIZATION PROBLEM

Avoid re-computing an expression that already been computed

At statement 4 the expression $a + b$ is always computed previously:

- either on line 2
- or on line 6

```
1: int compute (int a, int b){  
2: int x = a + b;  
3: int y = a * b;  
4: while(y > a + b){  
5:   a = a + 1;  
6:     x = a + b;}  
7: return x;}
```

OPTIMIZATION PROBLEM

Decision problem

Is an expression used in a statement S_1 must have been already computed previously and not modified later on all paths leading from where it was computed to S_1 .

- @2 $a + b$ is used, but has not been computed
- @3 $a * b$ is used, but has not been computed
- @6 $a + b$ has been computed, but modified on line 5

```
1: int compute (int a, int b){  
2: int x = a + b;  
3: int y = a * b;  
4: while(y > a + b){  
5:   a = a + 1;  
6:     x = a + b;}  
7: return x;}
```

AVAILABLE EXPRESSION ANALYSIS

What data to propagate?

- a set of expressions over variables and integers
- initialized
 - $OUT[stmt] = \{\}$
 - $IN[stmt] = \{\}$
- transfer function for a statement $s: lhs = rhs1 + rhs2$
 1. remove all expressions from $IN[s]$ with lhs value in it.
 2. if $lhs \neq rhs1$ and $lhs \neq rhs2$, then add $rhs1 + rhs2$ to the previous result.
- transfer function for a conditional statement $c: if (lhs < rhs1 + rhs2)$
 - Add $(rhs1 + rhs2)$ expression to $IN[c]$.
- when two flows are merged – do intersection of all expressions
 - to remain in the flow an expression must be computed on all paths, i.e., on all incoming flows.

AVAILABLE EXPRESSION

Formally defined:

- For an assignment statement $x = a$;
 - x is a lhs
 - a is an expression, which can contain a set of subexpressions defined by a function $AExpr(a)$
 - An $a' \in AExpr(a)$ can have constants and free variables, we define the latter by $FV(a')$
 - $OUT[x = a] = (IN[x = a] \setminus \{a' \in IN[x = a] \mid x \in FV(a')\}) \cup \{a' \in AExpr(a) \mid x \notin FV(a')\}$
- For an expression in a conditional statement $if(b)$
 - $OUT[b] = IN[b] \cup AExpr(b)$
- Other statements:
 - $OUT[b] = IN[b]$
- Combining flows:
 - $IN[s] = \cap \{OUT[s'] \mid s' \in CFG.pred(s)\}$

```
1: int compute (int a, int b){  
2: int x = a + b;  
3: int y = a * b;  
4: while(y > a + b){  
5:   a = a + 1;  
6:   x = a + b;}  
7: return x;}
```

REACHING DEFINITIONS

- Formally defined
- Let *Labels* be a set of line numbers of assignment statements
- For an assignment statement $l: x = a;$
 - $OUT[l: x = a] = (IN[l: x = a] \setminus \{n@x \mid \forall n \in Labels\}) \cup \{x@l\}$
- Other statements:
 - $OUT[b] = IN[b]$
- Combining flows:
 - $IN[s] = \cup \{ OUT[s'] \mid s' \in CFG.pred(s) \}$

OPTIMIZATION PROBLEM

Keep a value of an expression that is guaranteed to be used, in a register.

At the beginning of 3, both $b-a$ and $a-b$ expression will be used.

We can precompute them before 3.

Hoisting optimization, or loop-invariant code motion.

Very Busy Expression analysis

```
1: int compute (int a, int b){  
2: int x;int y;  
3: if(a > b){  
4:   x = b - a;  
5:     y = a - b;  
6: }else {  
7:   y = b - a;  
8:   x = a - b;}  
9: return x/y;}
```

VERY BUSY EXPRESSIONS

Start from the end of CFG, i.e., CFG^R

- For an assignment statement $x = a$:
 - if lhs appears in expressions of the flow, i.e., $x \in FV(a')$ then remove them
 - add expressions of rhs to the flow, i.e., $a' \in AExp(a)$
 - $OUT[x = a] = (IN[x = a] \setminus \{a' \in IN[x = a] \mid x \in FV(a')\}) \cup \{a' \in AExp(a) \mid x \notin FV(a')\}$
- For an expression in a conditional statement $if(b)$:
 - $OUT[b] = IN[b] \cup AExpr(b)$
- Other statements:
 - $OUT[b] = IN[b]$
- Combining flows:
 - Intersection since it must be used on all paths coming up
 - $IN[s] = \cap \{OUT[s'] \mid s' \in CFG^R.pred(s)\}$

```
1: int compute (int a, int b){  
2: int x;int y;  
3: if(a > b){  
4:   x = b - a;  
5:       y = a - b;  
6: }else {  
7:   y = b - a;  
8:   x = a - b;}  
9: return x/y;}
```


OPTIMIZATION PROBLEM

Remove variable assignments that are not used later in the code, or remove from registers values that are not later used in the code.

At a given location what variables are not later used?

At 9 z is used (but not x)

Analysis on the reverse CFG, CFG^R

Live Variable analysis

```
1: int compute (int z){  
2: int x = 2;  
3: int y = 4;  
4: int x = 1;  
4: if(y > x){  
5:     z = y;  
6: }else {  
7:     z = y * y;}  
8: x = z;  
9: return z;}
```

LIVE VARIABLES

- Formally defined
- For an assignment statement $l: x = a$;
 - $OUT[l: x = a] = (IN[l: x = a] \setminus \{x\}) \cup FV(a)$
- For an expression in a conditional statement $if(b)$:
 - $OUT[b] = IN[b] \cup AExpr(b)$
- Other statements:
 - $OUT[b] = IN[b]$
- Combining flows:
 - $IN[s] = \cup \{ OUT[s'] \mid s' \in CFG^R.pred(s) \}$

```
1: int compute (int z){  
2: int x = 2;  
3: int y = 4;  
4: int x = 1;  
4: if(y > x){  
5:         z = y;  
6: }else {  
7:   z = y * y;}  
8: x = z;  
9: return z;}
```



NEXT TIME

Defining data-flow framework to instantiate any data-flow analysis